

Refatorar

Nenhuma funcionalidade nova hoje. No fim da aula o jogo faz exatamente o que fazia no começo — e é isso que torna a aula difícil.

Continuação direta da IA de inimigo.

Você precisa do jogo com `agir` funcionando nos seis inimigos antes de começar.

Onde você está

Faz dez aulas que você acrescenta coisa ao mesmo jogo. Herança, encapsulamento, composição, itens, fases, progressão, efeitos temporários, habilidades especiais, inteligência de inimigo. Cada aula pediu para você **escrever** alguma coisa.

Hoje é a primeira aula que pede para você **apagar**.

O seu jogo funciona. Ele roda do começo ao fim, seis fases, sem erro na tela. E mesmo assim ele tem problema — só que o problema não aparece jogando. Aparece na próxima vez que alguém precisar mexer nele.

O que é refatorar

Refatorar é mudar a **forma** do código sem mudar o **comportamento** dele. Você reorganiza por dentro, e por fora ninguém percebe diferença nenhuma.

A REGRA DO DIA

Se o jogo passou a fazer algo diferente, você não refatorou: você quebrou. **Mesma entrada, mesma saída.** Essa é a única medida de sucesso de hoje.

Isso soa óbvio e é a parte mais difícil da aula. A tentação de "já que estou aqui, deixa eu melhorar o dano do Troll" vai aparecer umas cinco vezes hoje. Toda vez que ela aparecer, anote a ideia num papel e não faça. Melhorar o jogo é outra aula.

Como saber que você não quebrou nada

Nós não temos teste automatizado. Então a conferência é no olho, e para ela funcionar você precisa fazer duas coisas que parecem bobas:

1. **Jogue antes.** Uma partida inteira, antes de tocar em qualquer linha. Preste atenção no que aparece na tela.
2. **Refatore de um capítulo por vez.** Termine o capítulo, jogue de novo, confira. Só então comece o próximo.

Cada capítulo termina com um bloco verde dizendo exatamente o que conferir. Se você fizer os cinco capítulos de uma vez e só então testar, e alguma coisa estiver errada, você não vai saber qual dos cinco causou. Aí a aula vira caça ao erro.

Guarde uma cópia. Antes de começar, copie a sua pasta `rpg/` inteira para `rpg-antes/`. Se você se perder no meio, é para lá que você volta. Não é vergonha nenhuma: é o que se faz.

A sua pasta

Continuam quatro arquivos. Hoje **nenhum arquivo novo** entra — e esse é um detalhe da aula, não um acaso.

```
rpg/  
  personagem.py    a classe Personagem  
  classes.py       Guerreiro, Mago e os seis inimigos  
  itens.py         poções, elixires e o inventário  
  main.py          fases, inventário, combate e progressão
```

O caminho

1 O mesmo código duas vezes

A linha do dano está copiada. Você extrai um método.

2 Nomes no lugar de números

O que é `0.3`? O que é `rolagem < 5`?

3 O `if` que cresce

Você já resolveu isso uma vez, sem saber o nome.

4 A guarda repetida

Quatro classes abrem a habilidade com o mesmo `if`.

5 Um objeto escondido

A duplicação que o capítulo 1 não consegue resolver.

Os capítulos estão em ordem de dificuldade, e o 5 depende do 1: ele resolve um caso que o capítulo 1 vai deixar em aberto de propósito.

Como cada capítulo funciona: problema, explicação, dois exemplos completos fora do RPG, e aí o exercício no seu jogo. Os exemplos não são enfeite — a técnica fica muito mais fácil de enxergar em código que você não escreveu.

Sumário

01 O mesmo código duas vezes

02 Nomes no lugar de números

03 O `if` que cresce

04 A guarda repetida

05 Um objeto escondido

AN Anexo A: erros que você vai ver

AN Anexo B: extras

AN Anexo C: checklist de entrega

O mesmo código duas vezes

PROBLEMA → EXPLICAÇÃO → EXEMPLO → EXEMPLO → EXERCÍCIO

O problema

Abra o `personagem.py` e o `classes.py` lado a lado. Procure a linha que sorteia o dano.

PERSONAGEM.PY · DENTRO DE PERSONAGEM.ATACAR

```
dano = random.randint(self.ataque - 3, self.ataque + 3)
```

CLASSES.PY · DENTRO DE GUERREIRO.ATACAR

```
dano = random.randint(self.ataque - 3, self.ataque + 3)
```

A mesma linha, letra por letra, em dois arquivos diferentes. Ninguém fez isso por burrice: na aula de herança o Guerreiro precisou de um ataque que nunca erra, e a forma mais rápida de conseguir isso foi copiar o corpo do método e apagar o sorteio de erro. Funcionou. E a linha do dano veio junto.

Agora imagine que amanhã a variação do dano deixe de ser 3 e passe a ser 5. Quantos lugares você precisa lembrar de mudar? Dois. E se você esquecer um, o jogo não dá erro — ele fica **sutilmente errado**, com o Guerreiro causando dano numa faixa e todo mundo em outra. Esse é o tipo de bug que ninguém acha.

O INCÔMODO

Toda vez que você copia e cola código, você cria uma promessa: "vou lembrar de mudar os dois". Você não vai.

A explicação: extrair método

A técnica se chama **extrair método** e ela tem três passos mecânicos:

1. Pegue o trecho repetido e dê um **nome** a ele.
2. Crie um método com esse nome, contendo o trecho.
3. Troque cada cópia por uma chamada ao método.

Repare no que o passo 1 tem de especial. Você não está só evitando repetição — está sendo obrigado a **dizer o que aquele código faz**. Um trecho que você não consegue nomear costuma ser um trecho que faz duas coisas ao mesmo tempo.

Exemplo 1: o boletim

Um sistema de notas calcula a média da turma em dois lugares: no boletim de cada aluno e no relatório do diretor.

ANTES

```
class Turma:
    def __init__(self, nome, notas):
        self.nome = nome
        self.notas = notas

    def boletim(self):
        soma = 0
        for nota in self.notas:
            soma += nota
        media = soma / len(self.notas)
        print("Turma", self.nome, "- média", media)

    def relatorio_diretor(self):
        soma = 0
        for nota in self.notas:
            soma += nota
        media = soma / len(self.notas)
        if media < 6:
            print("Turma", self.nome, "precisa de reforço")
```

Sete linhas iguais, duas vezes. Extraia:

DEPOIS

```
class Turma:
    def __init__(self, nome, notas):
        self.nome = nome
        self.notas = notas

    def media(self):
        soma = 0
        for nota in self.notas:
            soma += nota
        return soma / len(self.notas)

    def boletim(self):
        print("Turma", self.nome, "- média", self.media())

    def relatorio_diretor(self):
        if self.media() < 6:
            print("Turma", self.nome, "precisa de reforço")
```

Os dois métodos que sobraram agora cabem numa linha cada, e dá para ler o que eles fazem sem parar para decifrar. O cálculo virou uma coisa com nome: `media`.

Repare que o `media` usa `return`, não `print`. Um método extraído quase sempre **devolve** um valor, porque quem chamou é que decide o que fazer com ele — um imprime, o outro compara. Se `media` imprimisse, o relatório do diretor imprimiria a média sem querer.

Exemplo 2: o carrinho de supermercado

Aqui a repetição não é literal — muda um pedacinho. É o caso mais comum e o mais fácil de errar.

ANTES

```
class Carrinho:
    def __init__(self):
        self.itens = []

    def adicionar(self, nome, preco, quantidade):
        self.itens.append({"nome": nome, "preco": preco, "qtd": quantidade})

    def total(self):
        soma = 0
        for item in self.itens:
            soma += item["preco"] * item["qtd"]
        return soma

    def total_com_desconto(self, porcentagem):
        soma = 0
        for item in self.itens:
            soma += item["preco"] * item["qtd"]
        return soma - soma * porcentagem / 100
```

O laço é idêntico; só o que se faz com o resultado muda. Então o laço é o que sai:

DEPOIS

```
def total(self):
    soma = 0
    for item in self.itens:
        soma += item["preco"] * item["qtd"]
    return soma

def total_com_desconto(self, porcentagem):
    soma = self.total()
    return soma - soma * porcentagem / 100
```

Nenhum método novo precisou nascer: o `total` já era o nome certo para aquele trecho, e o `total_com_desconto` passou a chamá-lo. Extrair método muitas vezes é só perceber

que o método já existe.

A PERGUNTA QUE RESOLVE

Antes de criar um método novo, olhe os que já existem e pergunte: **algum deles já faz exatamente esse pedaço?** Se sim, chame. Duplicação some tanto criando código quanto reaproveitando.

Exercício 1: o dano sorteado

ARQUIVOS: `PERSONAGEM.PY` E `CLASSES.PY`

Faça:

1. Em `Personagem`, crie o método `calcular_dano(self)`. Ele não imprime nada: **devolve** o valor sorteado.
2. Em `Personagem.atacar`, troque o sorteio pela chamada ao novo método.
3. Em `Guerreiro.atacar`, faça a mesma troca.
4. Rode o jogo e ataque com o Guerreiro e com um inimigo. O dano continua saindo na mesma faixa.

CONFIRA JOGANDO

Comece uma partida e ataque o Goblin três vezes. Os números de dano precisam continuar na mesma faixa de antes — nem sempre iguais, porque é sorteio, mas nunca um dano gigante nem zero.

Confira também que o Guerreiro **continua nunca errando** e que os outros personagens continuam errando de vez em quando. Se o Guerreiro passou a errar, você extraiu demais: levou junto o sorteio do erro, que não era duplicado.

Exercício 2: a recompensa da fase

ARQUIVO: `MAIN.PY`

Olhe o `main()`. Depois de vencer uma fase acontecem quatro coisas seguidas: subir de nível, curar 50, ganhar o prêmio, e avisar na tela. Isso é um bloco com identidade própria dentro de uma função que já é longa.

1. Crie a função `recompensar(jogador, fase)`, fora do `main`, no nível do arquivo.
2. Mova as quatro coisas para dentro dela, sem mudar a ordem nem o texto dos `print`.
3. No `main`, chame `recompensar(jogador, fase)` no lugar do bloco.

Aqui não havia duplicação nenhuma — o bloco aparecia uma vez só. Extraí-lo mesmo assim vale a pena porque o `main` encolhe e passa a se ler como um resumo: joga a fase, se perdeu acaba, se ganhou recompensa. Extrair método também serve para dar nome, não só para eliminar cópia.

CONFIRA JOGANDO

Vença a primeira fase. Tem que aparecer, na mesma ordem de antes: subiu para o nível 2, descansou e recuperou 50 de vida, e o Goblin deixou cair a poção. Nenhuma linha a mais, nenhuma a menos.

Terminou o capítulo 1? Os extras `E1.1` e `E1.2` do anexo B dependem dele.

Nomes no lugar de números

PROBLEMA → EXPLICAÇÃO → EXEMPLO → EXEMPLO → EXERCÍCIO

O problema

Abra o `classes.py` no `agir` do Goblin:

```
if self.vida < self.vida_maxima * 0.3:
    self.defender()
```

Você escreveu isso ontem, então hoje o `0.3` ainda faz sentido. Daqui a três semanas, não vai fazer. E para quem nunca viu o código, não faz sentido nenhum agora.

O jogo está cheio disso:

O QUE ESTÁ ESCRITO	O QUE SIGNIFICA
<code>random.randint(1, 20)</code>	um dado de 20 lados
<code>if rolagem < 5</code>	rolar 1 a 4 é errar o ataque
<code>self.vida_maxima * 0.3</code>	o Goblin considera que está em apuros
<code>self.curar(50)</code>	o descanso entre fases
<code>self.aplicar_bonus_ataque_temporario(6, 3)</code>	a fúria do Orc, e quanto ela dura

Nenhum desses números está errado. O problema é que eles são **mudos**. Um número solto no meio do código tem até nome próprio na profissão: número mágico. Mágico porque aparece do nada e ninguém sabe de onde veio.

A explicação: constante nomeada

A técnica é a mais barata do dia: dê um nome ao número e use o nome.

Em Python, constantes se escrevem em **MAIÚSCULAS**, no topo do arquivo, fora de qualquer classe. Isso é convenção, não regra da linguagem — o Python deixa você mudar o valor, mas o nome em maiúscula avisa a quem lê: isto aqui é para ficar parado.

```
LADOS_DO_DADO = 20
ROLAGEM_MINIMA_PARA_ACERTAR = 5
```

Ganha-se duas coisas ao mesmo tempo, e a segunda é a que importa mais:

- Se o valor mudar, muda num lugar só.
- **O código passa a explicar a si mesmo.** `if rolagem < ROLAGEM_MINIMA_PARA_ACERTAR` não precisa de comentário. `if rolagem < 5` precisa.

Um número não precisa de nome quando ele é **o próprio conceito**. Em `soma / len(notas)` ou `turnos - 1`, ninguém vai criar `UM = 1`. A pergunta certa não é "isto é um número?", é "**alguém que lê isto sabe por que é esse valor?**".

Exemplo 1: o elevador

ANTES

```
class Elevador:
    def __init__(self):
        self.pessoas = 0
        self.andar = 0

    def entrar(self, quantidade):
        if self.pessoas + quantidade > 8:
            print("Excesso de peso")
            return
        self.pessoas += quantidade

    def subir(self):
        if self.andar >= 12:
            print("Último andar")
            return
        self.andar += 1

    def pode_partir(self):
        return self.pessoas <= 8
```

Repare no `8`: ele aparece duas vezes. Se a capacidade mudar para 10 e você trocar só um, o elevador aceita 10 pessoas e depois se recusa a partir. Um bug que só acontece com o elevador cheio.

DEPOIS

```

CAPACIDADE_MAXIMA = 8
ULTIMO_ANDAR = 12

class Elevador:
    def __init__(self):
        self.pessoas = 0
        self.andar = 0

    def entrar(self, quantidade):
        if self.pessoas + quantidade > CAPACIDADE_MAXIMA:
            print("Excesso de peso")
            return
        self.pessoas += quantidade

    def subir(self):
        if self.andar >= ULTIMO_ANDAR:
            print("Último andar")
            return
        self.andar += 1

    def pode_partir(self):
        return self.pessoas <= CAPACIDADE_MAXIMA

```

Agora é impossível mudar a capacidade pela metade.

Exemplo 2: o semáforo

Este exemplo mostra o caso em que a constante não é um número, e sim um texto.

ANTES

```

class Semaforo:
    def __init__(self):
        self.cor = "vermelho"
        self.segundos = 0

    def passar_segundo(self):
        self.segundos += 1

        if self.cor == "verde" and self.segundos >= 30:
            self.cor = "amarelo"
            self.segundos = 0
        elif self.cor == "amarelo" and self.segundos >= 5:
            self.cor = "vermelho"
            self.segundos = 0
        elif self.cor == "vermelho" and self.segundos >= 25:
            self.cor = "verde"
            self.segundos = 0

```

Os textos "verde", "amarelo" e "vermelho" estão escritos à mão seis vezes. Se você digitar "vermelhoo" em um deles, o Python não reclama: o `elif` simplesmente nunca é verdadeiro e o semáforo trava numa cor. Erro silencioso, de novo.

DEPOIS

```
VERDE = "verde"
AMARELO = "amarelo"
VERMELHO = "vermelho"

DURACAO_VERDE = 30
DURACAO_AMARELO = 5
DURACAO_VERMELHO = 25

class Semaforo:
    def __init__(self):
        self.cor = VERMELHO
        self.segundos = 0

    def passar_segundo(self):
        self.segundos += 1

        if self.cor == VERDE and self.segundos >= DURACAO_VERDE:
            self.trocar(AMARELO)
        elif self.cor == AMARELO and self.segundos >= DURACAO_AMARELO:
            self.trocar(VERMELHO)
        elif self.cor == VERMELHO and self.segundos >= DURACAO_VERMELHO:
            self.trocar(VERDE)

    def trocar(self, cor):
        self.cor = cor
        self.segundos = 0
```

Agora um nome digitado errado vira `NameError` na hora, em vez de um semáforo travado às três da manhã. E aproveitamos para extrair o `trocar`, que estava copiado três vezes — é o capítulo 1 aparecendo de novo. As técnicas se acumulam.

Exercício 3: batizar os números do jogo

ARQUIVOS: `PERSONAGEM.PY` E `CLASSES.PY`

Faça, no `personagem.py`:

1. Crie `LADOS_DO_DADO = 20` e `ROLAGEM_MINIMA_PARA_ACERTAR = 5` no topo do arquivo, logo abaixo dos `import`.
2. Use as duas no `atacar`.
3. Crie `VARIACAO_DO_DANO = 3` e use no `calcular_dano` que você fez no capítulo 1.

Agora no `classes.py`, nos `agir` que você escreveu ontem:

1. O `0.3` do Goblin vira `VIDA_BAIXA_GOBLIN = 0.3`.
2. O `0.5` do Orc e o `0.4` do Troll ganham nomes no mesmo estilo.
3. A fúria do Orc — o `6` e o `3` — vira `FORCA_DA_FURIA` e `DURACAO_DA_FURIA`.
4. Faça o mesmo com o endurecimento do Troll.

O QUE NÃO FAZER

Não crie constante para os números do `super().__init__("Goblin", 40, 8, 2, 0)`. Ali o problema é outro: não é que os números não tenham nome, é que a **chamada** não diz qual é qual. Isso se resolve de um jeito diferente, e está no extra `E2.2`.

CONFIRA JOGANDO

Jogue até a segunda fase. O Goblin tem que continuar se defendendo quando está quase morrendo, e o Orc tem que continuar entrando em fúria com a mensagem de sempre.

Depois faça um teste de propósito: mude `ROLAGEM_MINIMA_PARA_ACERTAR` para `15` e jogue. Quase todo mundo tem que passar a errar. Isso prova que a constante está realmente sendo usada, e não que sobrou um `5` escondido. **Volte para 5 depois.**

Terminou o capítulo 2? Os extras `E2.1` e `E2.2` dependem dele.

O `if` que cresce

PROBLEMA → EXPLICAÇÃO → EXEMPLO → EXEMPLO → EXERCÍCIO

O problema

Este capítulo tem uma vantagem sobre os outros: você já resolveu esse problema uma vez, sem saber que tinha nome. Vamos usar a sua própria solução como resposta.

Olhe o `turno_do_jogador`, no `main.py`:

```
if escolha == "1":
    jogador.atacar(inimigo)
elif escolha == "2":
    jogador.defender()
elif escolha == "3":
    abrir_mochila(jogador, inimigo)
elif escolha == "4":
    jogador.habilidade_especial(inimigo)
else:
    print("Opção inválida")
```

Funciona. Mas conte o que acontece quando você quiser acrescentar a opção "5 - fugir": você mexe no `print` do menu, escreve mais um `elif`, e reza para os dois estarem combinando. O menu e as ações são **duas listas separadas que precisam concordar**, e nada no código obriga elas a concordar.

Agora olhe o topo do mesmo arquivo, onde você resolveu isso direito:

```
CLASSES = {
    "1": Guerreiro,
    "2": Mago,
}
```

E o `escolher_personagem` que usa esse dicionário:

```
for tecla, classe in CLASSES.items():
    print(tecla, "-", classe.__name__)

escolha = input("Opção: ")
while escolha not in CLASSES:
    print("Opção inválida")
    escolha = input("Opção: ")
```

Repare no que essa função **não** tem: nenhum `if` comparando com `"1"`. Ela imprime o menu a partir do dicionário e valida a escolha a partir do mesmo dicionário. Acrescentar a classe Arqueiro é uma linha, num lugar só, e o menu se atualiza sozinho.

A explicação: trocar condicional por tabela

A ideia: quando uma cadeia de `if/elif` só serve para **escolher entre coisas do mesmo tipo**, ela pode virar um dicionário — e a cadeia some.

Isso funciona porque em Python uma função é um valor como outro qualquer. Você pode guardar uma função dentro de um dicionário, tirar de lá e chamar depois:

```
def cumprimentar():
    print("Olá")

acoes = {"a": cumprimentar}

acao = acoes["a"] # guardou a função, sem chamar
acao()           # só agora ela executa
```

O DETALHE QUE DERRUBA TODO MUNDO

`acoes["a"]` pega a função. `acoes["a"]()` pega a função **e chama**. Se você escrever os parênteses dentro do dicionário — `{"a": cumprimentar()}` — a função executa na hora em que o dicionário é criado, e o que fica guardado é o `None` que ela devolveu.

E a cadeia não some sempre. Ela só some quando os ramos são **intercambiáveis**: mesma pergunta, respostas do mesmo formato. Se cada ramo faz uma coisa de natureza diferente, o dicionário fica pior que o `if`.

Exemplo 1: o caixa eletrônico

ANTES

```

class Caixa:
    def __init__(self, saldo):
        self.saldo = saldo

    def menu(self):
        print("1 - Ver saldo")
        print("2 - Sacar")
        print("3 - Depositar")
        print("4 - Extrato")

        escolha = input("Opção: ")

        if escolha == "1":
            self.ver_saldo()
        elif escolha == "2":
            self.sacar()
        elif escolha == "3":
            self.depositar()
        elif escolha == "4":
            self.extrato()
        else:
            print("Opção inválida")

```

O nome de cada opção aparece duas vezes: uma no `print`, outra no `elif`. Junte os dois num lugar só.

DEPOIS

```

class Caixa:
    def __init__(self, saldo):
        self.saldo = saldo

    def menu(self):
        opcoes = {
            "1": ("Ver saldo", self.ver_saldo),
            "2": ("Sacar", self.sacar),
            "3": ("Depositar", self.depositar),
            "4": ("Extrato", self.extrato),
        }

        for tecla, (rotulo, _) in opcoes.items():
            print(tecla, "-", rotulo)

        escolha = input("Opção: ")

        if escolha not in opcoes:
            print("Opção inválida")
            return

        rotulo, acao = opcoes[escolha]
        acao()

```

O texto e a ação viraram um par. Não existe mais como o menu oferecer "Extrato" e o programa chamar outra coisa — eles estão na mesma linha.

O `_` no `for tecla, (rotulo, _) in ...` é um nome de variável comum, mas com um combinado por trás: **"eu recebi isto e não vou usar"**. O Python não trata o `_` de forma especial; quem trata é quem lê.

Exemplo 2: a catraca do ônibus

Nem toda cadeia vira tabela de funções. Às vezes o que varia é só um **valor**, e aí o dicionário fica ainda mais simples.

ANTES

```
class Catraca:
    def cobrar(self, tipo_de_passagem):
        if tipo_de_passagem == "inteira":
            valor = 4.50
        elif tipo_de_passagem == "estudante":
            valor = 2.25
        elif tipo_de_passagem == "idoso":
            valor = 0.00
        elif tipo_de_passagem == "professor":
            valor = 2.25
        else:
            print("Passagem desconhecida")
            return

        print("Cobrado:", valor)
```

DEPOIS

```
TARIFAS = {
    "inteira": 4.50,
    "estudante": 2.25,
    "idoso": 0.00,
    "professor": 2.25,
}

class Catraca:
    def cobrar(self, tipo_de_passagem):
        if tipo_de_passagem not in TARIFAS:
            print("Passagem desconhecida")
            return

        print("Cobrado:", TARIFAS[tipo_de_passagem])
```

Quinze linhas viraram cinco, e a tabela de tarifas passou a ser uma coisa que existe sozinha — dá para imprimir, dá para conferir, e um dia dá para ler de um arquivo sem tocar na classe.

QUANDO NÃO FAZER

Se os ramos do `if` fazem coisas de **formatos diferentes** — um recebe dois parâmetros, outro nenhum, outro devolve valor — o dicionário obriga você a inventar remendos para uniformizar. Nesse caso o `if` honesto é melhor. A tabela é para quando os ramos já são parecidos.

Exercício 4: o menu do turno

ARQUIVO: MAIN.PY

Antes de escrever, repare num detalhe: as quatro ações do turno **não têm a mesma assinatura**. Duas recebem o inimigo, uma não recebe nada, e `abrir_mochila` recebe os dois.

```
jogador.atacar(inimigo)           # recebe o inimigo
jogador.defender()                # nao recebe nada
abrir_mochila(jogador, inimigo)    # recebe os dois
jogador.habilidade_especial(inimigo) # recebe o inimigo
```

Pelo aviso do quadro anterior, isso é exatamente o caso em que a tabela atrapalha. Só que aqui dá para uniformizar sem remendo: faça **todas** receberem os dois.

Faça:

1. Dentro de `turno_do_jogador`, crie quatro funções pequenas — ou use `lambda`, se preferir — que recebam `jogador` e `inimigo` e chamem a ação certa, ignorando o parâmetro que não usarem.
2. Monte o dicionário `ACOES` no formato `"tecla": (rótulo, função)`, como no caixa eletrônico.
3. Imprima o menu a partir do dicionário.
4. Valide com `if escolha not in ACOES` e chame a função escolhida com `(jogador, inimigo)`.

O texto das quatro opções tem que continuar igualzinho ao de antes, inclusive o `"\n1 - Atacar"` com a quebra de linha no começo.

CONFIRA JOGANDO

Entre num combate e use as quatro opções, uma de cada vez: ataque, defenda, abra a mochila (e volte com `0`), e use a habilidade especial. As quatro têm que se comportar exatamente como antes.

Digite `9` e depois `abc`. Nos dois casos tem que aparecer "Opção inválida" e o jogo continuar — sem travar e sem o inimigo perder a vez.

Por último, o teste que vale a aula: acrescente a opção `"5"` com o rótulo "Não fazer nada" e uma função que só imprime. Você tem que ter mexido em **um lugar só**, e o menu tem que passar a mostrar cinco opções sozinho. Depois apague.

Terminou o capítulo 3? O extra `E3.1` depende dele.

A guarda repetida

PROBLEMA → EXPLICAÇÃO → EXEMPLO → EXEMPLO → EXERCÍCIO

O problema

Quatro classes têm habilidade especial: Guerreiro, Mago, Esqueleto e Dragão. Abra as quatro e olhe só a primeira metade de cada `habilidade_especial`.

```
# Guerreiro
if not self.habilidade_disponivel():
    print("O golpe esmagador ainda se recompõe. Faltam",
          self._recarga_habilidade, "turno(s)")
    return

# Mago
if not self.habilidade_disponivel():
    print("A bola de fogo ainda se forma. Faltam",
          self._recarga_habilidade, "turno(s)")
    return

# Esqueleto
if not self.habilidade_disponivel():
    print("Os ossos ainda se recompõem. Faltam",
          self._recarga_habilidade, "turno(s)")
    return
```

A mesma estrutura quatro vezes: **confere a recarga, avisa, desiste**. Só a frase muda. E no fim de cada método tem outra repetição, o `self.iniciar_recarga(...)`.

O capítulo 1 não resolve isso. Extrair a guarda para um método não adianta, porque o `return` precisa sair do `habilidade_especial`, e um `return` dentro do método extraído só sai dele mesmo.

Pior: a repetição é uma **armadilha**. Amanhã você cria a Harpia com habilidade especial, esquece a guarda, e ela ataca com o golpe especial todo turno. Nenhum erro na tela. O jogo só fica desbalanceado, e você vai culpar os números do dano.

O INCÔMODO

Quando escrever uma classe nova exige **lembrar** de repetir um pedaço, mais cedo ou mais tarde alguém esquece. Código que depende de memória é código mal desenhado.

A explicação: a mãe cuida do ritual, a filha faz o efeito

A técnica se chama **método modelo**, e é a mais bonita do dia porque usa duas coisas que você já sabe — herança e sobrescrita — para um fim novo.

A ideia: separe o que é **igual em todas** do que é **diferente em cada uma**. O igual sobe para a mãe; o diferente vira um método que só a filha escreve.

No nosso caso:

O QUÊ	ONDE FICA
Conferir a recarga	igual em todas → sobe para Personagem
Avisar que falta recarga	igual em todas → sobe para Personagem
Iniciar a recarga depois	igual em todas → sobe para Personagem
O efeito da habilidade	diferente em cada uma → fica na filha
Quantos turnos de recarga	diferente em cada uma → fica na filha

A mãe passa a ser dona do **roteiro**. A filha preenche os buracos. Ninguém consegue esquecer a guarda, porque a guarda não está mais na mão da filha.

Exemplo 1: a impressora

ANTES

```

class Impressora:
    def __init__(self):
        self.papel = 10
        self.tinta = 100

class ImpressoraTexto(Impressora):
    def imprimir(self, documento):
        if self.papel == 0:
            print("Sem papel")
            return
        if self.tinta < 5:
            print("Sem tinta")
            return

        print("Imprimindo texto:", documento)
        self.papel -= 1
        self.tinta -= 5

class ImpressoraFoto(Impressora):
    def imprimir(self, documento):
        if self.papel == 0:
            print("Sem papel")
            return
        if self.tinta < 5:
            print("Sem tinta")
            return

        print("Imprimindo foto em alta qualidade:", documento)
        self.papel -= 1
        self.tinta -= 30

```

As duas conferem papel, conferem tinta, imprimem, e descontam. Só a linha do meio e o gasto de tinta mudam.

DEPOIS

```

class Impressora:
    def __init__(self):
        self.papel = 10
        self.tinta = 100

    def imprimir(self, documento):
        # o roteiro, igual para todas
        if self.papel == 0:
            print("Sem papel")
            return
        if self.tinta < self.gasto_de_tinta():
            print("Sem tinta")
            return

        self.desenhar(documento)
        self.papel -= 1
        self.tinta -= self.gasto_de_tinta()

    def gasto_de_tinta(self):
        return 5

    def desenhar(self, documento):
        print("Imprimindo:", documento)

class ImpressoraTexto(Impressora):
    def desenhar(self, documento):
        print("Imprimindo texto:", documento)

class ImpressoraFoto(Impressora):
    def gasto_de_tinta(self):
        return 30

    def desenhar(self, documento):
        print("Imprimindo foto em alta qualidade:", documento)

```

As filhas encolheram para duas e cinco linhas. E repare no que ficou impossível: **não dá mais para escrever uma impressora que esquece de conferir o papel**. A verificação não está mais ao alcance de quem escreve a filha.

Note que `ImpressoraTexto` não sobrescreveu `gasto_de_tinta`. Ela herdou o 5 da mãe. Um método modelo pode ter buracos com **valor padrão** — a filha só preenche o que for diferente nela.

Exemplo 2: o empréstimo da biblioteca

Aqui o roteiro tem uma parte **depois** do trecho variável, que é o caso do nosso RPG.

DEPOIS (JÁ REFATORADO)

```

class Emprestimo:
    def __init__(self, titulo):
        self.titulo = titulo
        self.emprestado = False

    def emprestar(self, pessoa):
        # 1. a guarda, igual para todos
        if self.emprestado:
            print(self.titulo, "já está emprestado")
            return

        # 2. o pedaço que muda
        self.registrar(pessoa)

        # 3. o fecho, igual para todos
        self.emprestado = True
        print("Devolver em", self.prazo_em_dias(), "dias")

    def prazo_em_dias(self):
        return 14

    def registrar(self, pessoa):
        print(self.titulo, "emprestado para", pessoa)

class Livro(Emprestimo):
    pass

class Revista(Emprestimo):
    def prazo_em_dias(self):
        return 7

class Raridade(Emprestimo):
    def prazo_em_dias(self):
        return 2

    def registrar(self, pessoa):
        print("ATENÇÃO: obra rara")
        print(self.titulo, "emprestado para", pessoa, "com assinatura")

```

A classe `Livro` é literalmente `pass`: o comportamento padrão da mãe já serve para ela. Esse é o sinal de que o roteiro ficou no lugar certo.

Exercício 5: a habilidade com roteiro

ARQUIVOS: `PERSONAGEM.PY` E `CLASSES.PY`

No `personagem.py`, dentro de `Personagem`:

1. Crie `usar_habilidade(self, alvo)`. Ele confere a recarga; se não estiver disponível, imprime o aviso usando `self.mensagem_de_recarga()` e dá `return`.

2. Se estiver disponível, ele chama `self.efeito_da_habilidade(alvo)` e depois `self.iniciar_recarga(self.turnos_de_recarga())`.
3. Crie os três buracos com valor padrão: `mensagem_de_recarga`, `turnos_de_recarga` devolvendo `3`, e `efeito_da_habilidade` imprimindo que o personagem não tem habilidade.

POR QUE O BURACO É A FRASE INTEIRA

A tentação é fazer o buraco devolver só o **nome** da habilidade, e a mãe montar a frase: `nome + "ainda se recompõe. Faltam..."`. Olhe as mensagens de hoje antes de decidir:

O Guerreiro diz que o golpe "**ainda se recompõe**". O Mago diz que a bola de fogo "**ainda se forma**". O Dragão diz que o sopro "**ainda se acumula**".

Não é só o nome que muda: o **verbo** muda. Se a mãe montar a frase, as três viram a mesma — e você mudou a saída da tela, que é justamente o que hoje proíbe. Por isso o buraco devolve a frase toda, até antes do ponto.

No `classes.py`, nas quatro classes que têm habilidade:

1. Apague o `habilidade_especial` inteiro de cada uma.
2. No lugar, escreva `efeito_da_habilidade` com **só o efeito** — sem a guarda no começo, sem o `iniciar_recarga` no fim.
3. Escreva `mensagem_de_recarga` em cada uma, com o texto exato que ela já imprimia.
4. Escreva `turnos_de_recarga` em cada uma — **menos** numa delas. O Dragão recarrega em 3 turnos, que é o padrão da mãe: ele não precisa escrever nada, igual à `ImpressoraTexto` do exemplo.

E onde a habilidade era chamada — no menu do turno e nos `agir` do Esqueleto e do Dragão — troque `habilidade_especial` por `usar_habilidade`.

CUIDADO COM O DRAGÃO

O `agir` do Dragão pergunta `if self.habilidade_disponivel()` antes de chamar. Agora que a guarda está dentro do `usar_habilidade`, essa pergunta virou **redundante** — mas não errada. Deixe como está e responda no extra `E4.1` se ela deveria sair.

CONFIRA JOGANDO

Com o Guerreiro: use a habilidade, confira o dano do Golpe Esmagador, e use de novo no turno seguinte. Tem que aparecer a mensagem de recarga, com o mesmo texto de antes e o número de turnos certo.

Refaça com o Mago. Espere a recarga acabar e confira que a habilidade volta a funcionar.

Chegue na Cripta e deixe o Esqueleto ficar com pouca vida: ele tem que remontar os ossos. No Covil, o Dragão tem que soltar o sopro e depois passar turnos sem soltar.

O teste que prova o desenho: crie uma classe de inimigo nova que só escreve

`efeito_da_habilidade`, sem escrever guarda nenhuma. Chame `usar_habilidade` duas vezes seguidas. Na segunda, a recarga tem que barrar sozinha. Depois apague a classe.

Terminou o capítulo 4? Os extras `E4.1` e `E4.2` dependem dele.

Um objeto escondido

PROBLEMA → EXPLICAÇÃO → EXEMPLO → EXEMPLO → EXERCÍCIO

O problema

Guardamos o pior para o fim. Abra o `personagem.py` e olhe estes dois métodos:

```
def aplicar_bonus_ataque_temporario(self, quantidade, turnos):
    if self._turnos_bonus_ataque > 0:
        self.ataque -= self._bonus_ataque

    self.ataque += quantidade
    self._bonus_ataque = quantidade
    self._turnos_bonus_ataque = turnos
    print(self.nome, "sente o ataque aumentar por um tempo")

def aplicar_bonus_defesa_temporario(self, quantidade, turnos):
    if self._turnos_bonus_defesa > 0:
        self.defesa -= self._bonus_defesa

    self.defesa += quantidade
    self._bonus_defesa = quantidade
    self._turnos_bonus_defesa = turnos
    print(self.nome, "sente a defesa aumentar por um tempo")
```

É o mesmo método duas vezes. Troque a palavra "ataque" por "defesa" e você tem o outro. E não para aí: dentro do `iniciar_turno` existe o mesmo par:

```
if self._turnos_bonus_ataque > 0:
    self._turnos_bonus_ataque -= 1
    if self._turnos_bonus_ataque == 0:
        self.ataque -= self._bonus_ataque
        self._bonus_ataque = 0
        print(self.nome, "sente o ataque voltar ao normal")

if self._turnos_bonus_defesa > 0:
    self._turnos_bonus_defesa -= 1
    if self._turnos_bonus_defesa == 0:
        self.defesa -= self._bonus_defesa
        self._bonus_defesa = 0
        print(self.nome, "sente a defesa voltar ao normal")
```

Uma ideia só — "um bônus que dura alguns turnos" — espalhada por **quatro atributos** e **quatro blocos de código**.

Por que o capítulo 1 não salva

Sua primeira reação deve ser: extraio um método e acabou. Tente, no papel. Você trava na hora de escrever a assinatura.

O método precisaria mexer ora em `self.ataque`, ora em `self.defesa`. Ora em `self._bonus_ataque`, ora em `self._bonus_defesa`. Não é o **valor** que muda entre as duas cópias: é o **nome do atributo**. E nome de atributo não se passa por parâmetro sem apelar para truques que deixariam o código muito pior do que está.

O DIAGNÓSTICO

Quando dois ou mais atributos **sempre andam juntos**, aparecem sempre no mesmo bloco e mudam sempre ao mesmo tempo, eles não são atributos soltos: são **um objeto que ninguém criou ainda**.

Olhe os nomes de novo: `_bonus_ataque` e `_turnos_bonus_ataque`. Um guarda "quanto" e o outro guarda "por quanto tempo". Eles nunca aparecem separados. Isso é um par — e um par com identidade própria tem nome: é um **bônus temporário**.

A explicação: extrair classe

A técnica se chama **extrair classe**. Você tira um grupo de atributos que andam juntos, cria uma classe com eles, e o objeto original passa a **ter** um dessa classe nova.

Isso é composição — a mesma coisa da parte 3, quando o `Personagem` ganhou um `Inventario`. Lá você compôs para criar uma funcionalidade nova. Aqui você compõe para **organizar o que já existe**. Mesma ferramenta, propósito diferente.

E tem um ganho de multiplicação: escreva a classe uma vez, use **duas**. Um objeto para o ataque, outro para a defesa. A duplicação some porque só existe um código, instanciado duas vezes.

Exemplo 1: o ar-condicionado

Um aparelho com dois temporizadores independentes: um para desligar sozinho, outro para o modo silencioso.

ANTES

```
class ArCondicionado:
    def __init__(self):
        self.ligado = False
        self.silencioso = False
        self.minutos_para_desligar = 0
        self.minutos_de_silencio = 0

    def passar_minuto(self):
        if self.minutos_para_desligar > 0:
            self.minutos_para_desligar -= 1
            if self.minutos_para_desligar == 0:
                self.ligado = False
                print("Desligando")

        if self.minutos_de_silencio > 0:
            self.minutos_de_silencio -= 1
            if self.minutos_de_silencio == 0:
                self.silencioso = False
                print("Voltando ao volume normal")
```

Igualzinho ao nosso `iniciar_turno`. O que está escondido ali é a ideia de "contagem regressiva".

DEPOIS

```

class Contagem:
    """Conta minutos para tras. Nao sabe o que acontece no fim."""

    def __init__(self):
        self.minutos = 0

    def comecar(self, minutos):
        self.minutos = minutos

    def ativa(self):
        return self.minutos > 0

    def passar_minuto(self):
        """Devolve True apenas no minuto em que a contagem chega ao fim."""
        if self.minutos == 0:
            return False

        self.minutos -= 1
        return self.minutos == 0

class ArCondicionado:
    def __init__(self):
        self.ligado = False
        self.silencioso = False
        self.desligamento = Contagem()
        self.silencio = Contagem()

    def passar_minuto(self):
        if self.desligamento.passar_minuto():
            self.ligado = False
            print("Desligando")

        if self.silencio.passar_minuto():
            self.silencioso = False
            print("Voltando ao volume normal")

```

Ainda existem dois blocos, e isso é proposital: **o que eles fazem no fim é genuinamente diferente** — um desliga o aparelho, o outro tira o silencioso. O que sumiu foi a contagem, que era a parte igual. Sobrou só a parte que de fato difere, e cada bloco caiu de cinco linhas para três.

Repare no comentário da classe `Contagem`: "não sabe o que acontece no fim". Isso é o que a torna reutilizável. Se ela soubesse desligar o ar-condicionado, não serviria para o modo silencioso — e muito menos para o nosso RPG.

Exemplo 2: o estoque da farmácia

Um caso em que o par de atributos é mais óbvio depois de nomeado do que antes.

ANTES

```

class Farmacia:
    def __init__(self):
        self.nome_generico = ""
        self.qtd_generico = 0
        self.nome_referencia = ""
        self.qtd_referencia = 0

    def vender_generico(self, quantidade):
        if self.qtd_generico < quantidade:
            print("Sem estoque de", self.nome_generico)
            return
        self.qtd_generico -= quantidade

    def vender_referencia(self, quantidade):
        if self.qtd_referencia < quantidade:
            print("Sem estoque de", self.nome_referencia)
            return
        self.qtd_referencia -= quantidade

```

DEPOIS

```

class Lote:
    def __init__(self, nome):
        self.nome = nome
        self.quantidade = 0

    def vender(self, quantidade):
        if self.quantidade < quantidade:
            print("Sem estoque de", self.nome)
            return False

        self.quantidade -= quantidade
        return True

class Farmacia:
    def __init__(self):
        self.generico = Lote("genérico")
        self.referencia = Lote("de referência")

```

A `Farmacia` ficou com **duas linhas**. E o ganho de verdade aparece depois: acrescentar um terceiro medicamento agora é uma linha, não quatro atributos e mais um método de venda.

Exercício 6: a classe BonusTemporario

ARQUIVO: PERSONAGEM.PY

Este é o exercício mais longo do dia. Faça em três etapas e jogue entre elas.

ETAPA 1 — ESCREVER A CLASSE

No topo do `personagem.py`, antes da classe `Personagem`, crie:

```

class BonusTemporario:
    """Guarda quanto e por quantos turnos. Nao sabe a que atributo pertence."""

    def __init__(self):
        self.quantidade = 0
        self.turnos = 0

    def ativo(self):
        # devolve True se ainda esta valendo

    def comecar(self, quantidade, turnos):
        # guarda os dois valores

    def passar_turno(self):
        # desconta um turno
        # devolve True SO no turno em que o bonus acaba

    def limpar(self):
        # zera a quantidade

```

Copie o modelo da `Contagem` do ar-condicionado. A diferença é que aqui a classe guarda também a **quantidade**, porque o `Personagem` vai precisar dela para desfazer o bônus.

ETAPA 2 — TROCAR OS QUATRO ATRIBUTOS POR DOIS OBJETOS

1. No `__init__` do `Personagem`, apague os quatro atributos `_bonus_*` e `_turnos_bonus_*`.
2. No lugar, crie `self.bonus_ataque = BonusTemporario()` e `self.bonus_defesa = BonusTemporario()`.
3. Reescreva os dois `aplicar_bonus_*_temporario` usando os objetos.
4. Reescreva os dois blocos do `iniciar_turno`. Cada um deve caber em quatro linhas.

Os novos atributos **não** começam com `_`. Isso é de propósito e vale uma conversa: o objeto `BonusTemporario` já protege o que importa por conta própria, com métodos. Deixá-lo público permite que o Orc pergunte `self.bonus_ataque.ativo()` — que é uma pergunta legítima, feita por um método, e não uma bisbilhotada num atributo privado.

ETAPA 3 — CONSERTAR QUEM LIA OS ATRIBUTOS ANTIGOS

Aqui está a parte que ensina a lição. Rode o jogo agora, antes de mexer no `classes.py`. Ele vai quebrar — e você já sabe onde, porque ontem o Orc e o Troll leram um atributo que não era deles:

```

if self._turnos_bonus_ataque == 0 and self.vida < self.vida_maxima * 0.5:

```

Troque nos dois inimigos por uma pergunta ao objeto:

```
if not self.bonus_ataque.ativo() and self.vida < VIDA_BAIXA_ORC:
```

O QUE ACONTECEU AQUI

O Orc quebrou porque estava lendo o **estado interno** da mãe em vez de fazer uma pergunta. Enquanto o dado estava solto em quatro atributos, não havia pergunta para fazer. Agora existe: `ativo()`.

Guarde o formato deste erro. Sempre que uma refatoração quebra algo em outra classe, ela está te mostrando onde o encapsulamento estava furado.

CONFIRA JOGANDO

Beba o Elixir de Fúria num combate. O ataque tem que subir com a mensagem de sempre, durar três turnos e voltar ao normal com a mensagem de sempre. Faça o mesmo com o Tônico de Pedra.

Beba dois elixires seguidos, sem esperar o primeiro acabar. O ataque tem que subir uma vez só, e não acumular — esse era o motivo daquele `if` no começo do método antigo. Se o seu ataque acumulou, você perdeu essa regra no caminho.

Chegue no Orc e deixe ele abaixo de metade da vida: ele tem que entrar em fúria uma vez só, e não a cada turno. Mesma conferência com o Troll.

Por último, olhe o seu `iniciar_turno`. Ele tem que estar visivelmente menor do que estava de manhã. Se não estiver, alguma coisa não foi para dentro da classe nova.

O que você fez hoje

TÉCNICA	O SINAL DE QUE ERA HORA DE USAR
Extrair método	O mesmo trecho aparece duas vezes.
Constante nomeada	Um número que só você sabe explicar.
Condicional → tabela	Um <code>if/elif</code> que cresce toda vez que o programa cresce.
Método modelo	Escrever uma classe nova exige lembrar de repetir um ritual.
Extrair classe	Atributos que sempre andam juntos.

Nenhuma delas é sobre o Python. Todas as cinco existem em Java, em C#, em JavaScript, com os mesmos nomes. O que você aprendeu hoje é a primeira coisa da matéria que não é da linguagem: é sobre **projeto**.

O QUE FICA

Nenhuma dessas técnicas é uma regra para seguir sempre. Cada uma tem um sinal que a chama, e você aprende a enxergar o sinal apanhando. Um código com pouca duplicação e nomes bons não é mais bonito: é mais **barato de mudar** — e é isso que vale, porque todo código vai ser mudado.

Erros que você vai ver

Refatorar produz uma família de erros bem diferente da de escrever código novo. Quase todos são "esqueci de trocar em algum lugar".

1. ATTRIBUTEERROR NUM ATRIBUTO QUE VOCÊ APAGOU

```
AttributeError: 'Orc' object has no attribute '_turnos_bonus_ataque'
```

Capítulo 5. Você trocou os atributos dentro do `Personagem` mas alguém de fora ainda lê o nome antigo. É esperado — está previsto na etapa 3 do exercício 6.

Para achar todos de uma vez, procure por `_bonus_` nos quatro arquivos antes de rodar. O que aparecer fora do `personagem.py` tem que virar chamada de método.

2. TYPEERROR: TAKES 2 POSITIONAL ARGUMENTS BUT 3 WERE GIVEN

```
TypeError: efeito_da_habilidade() takes 2 positional arguments but 3 were given
```

Capítulo 4. A mãe chama `self.efeito_da_habilidade(alvo)` com um parâmetro; a sua filha declarou o método com outro número. As assinaturas do buraco e do preenchimento têm que bater exatamente.

3. O MENU IMPRIME NONE DEPOIS DE CADA AÇÃO

Sem erro na tela: a ação funciona e aparece um `None` solto embaixo.

Capítulo 3. Você escreveu `print(acao())` em vez de só `acao()`. As ações não devolvem nada, e o `print` imprime esse nada.

4. A AÇÃO ACONTECE SOZINHA, ANTES DE VOCÊ ESCOLHER

Você monta o menu e o personagem já ataca, antes de aparecer a pergunta.

Capítulo 3, o erro dos parênteses. Você escreveu `{"1": jogador.atacar(inimigo)}`. Isso **chama** o ataque na hora de montar o dicionário. O que fica guardado é o `None` devolvido.

ERRADO

```
ACOES = {"1": ("Atacar", atacar(jogador, inimigo))}
```

CERTO

```
ACOES = {"1": ("Atacar", atacar)}
```

5. A HABILIDADE ESPECIAL PAROU DE ENTRAR EM RECARGA

Sem erro: a habilidade funciona todo turno, sem limite.

Capítulo 4. Ao apagar o `habilidade_especial` antigo, você levou junto o `iniciar_recarga` e esqueceu de colocá-lo no `usar_habilidade` da mãe. Confira que a mãe chama `self.iniciar_recarga(self.turnos_de_recarga())` **depois** do efeito.

6. O BÔNUS DO ELIXIR ACUMULA

Sem erro: beba dois elixires seguidos e o ataque sobe duas vezes.

Capítulo 5. O método antigo começava desfazendo o bônus anterior se ainda estivesse valendo. Ao reescrever com o objeto, essa parte se perdeu. O `aplicar_bonus_ataque_temporario` novo ainda precisa perguntar `if self.bonus_ataque.ativo()` e descontar antes de aplicar o novo.

7. O BÔNUS NUNCA ACABA

Sem erro: o ataque sobe e fica alto para sempre.

Capítulo 5. O seu `passar_turno` está devolvendo `True` na hora errada — ou nunca. Ele devolve `True` **apenas** no turno em que os turnos chegam a zero; nos demais, `False`. Se ele devolver `self.turnos == 0` logo na primeira linha, ele devolve `True` sempre que estiver parado, e o desconto acontece toda hora.

8. O JOGO ESTÁ CERTO MAS O TEXTO MUDOU

Nada quebrou, tudo funciona — só que uma mensagem saiu diferente da de antes.

Isso é falha de refatoração igual às outras. Mesmo comportamento inclui **mesma saída na tela**. Volte e recupere o texto exato, com a mesma pontuação e os mesmos acentos.

Extras

Tudo aqui é opcional. Nenhum capítulo depende destes itens.

E1.1 O ATAQUE DO TROLL

Depende de: capítulo 1.

O Troll sorteia o dano numa faixa diferente de todo mundo: `randint(self.ataque, self.ataque + 10)`.

Faça o `calcular_dano` aceitar a faixa como parâmetro, com valor padrão igual ao de hoje, e o Troll passar a usá-lo. Responda: o método ficou mais útil ou só mais complicado?

E1.2 EXTRAIR DEMAIS

Depende de: capítulo 1.

Este extra é de opinião, e não tem resposta certa. Pegue o `combate` do `main.py` e extraia o corpo do `while` para uma função `um_turno(jogador, inimigo)`.

Depois olhe o resultado e decida: ficou mais fácil ou mais difícil de ler? Escreva três linhas justificando. Extrair método também tem exagero.

E2.1 A TABELA DE BALANCEAMENTO

Depende de: capítulo 2.

Junte todas as constantes que você criou no `classes.py` num bloco só, no topo, separadas por comentário.

Agora tente balancear o jogo mexendo **só** nesse bloco: deixe o Goblin mais covarde e o Orc mais agressivo. Repare que você não precisou entrar em método nenhum.

E2.2 A CHAMADA QUE NÃO DIZ NADA

Depende de: capítulo 2.

Olhe `super().__init__("Goblin", 40, 8, 2, 0)`. Sem abrir o `Personagem`, diga qual número é a defesa.

Python permite nomear os argumentos na chamada: `super().__init__("Goblin", vida=40, ataque=8, defesa=2, pcoes=0)`. Faça isso nos seis inimigos e compare.

Responda: por que isso resolve melhor que criar constantes `VIDA_DO_GOBLIN = 40`?

E3.1 A TABELA SOBE DE NÍVEL

Depende de: capítulo 3.

Tire o dicionário `ACOES` de dentro do `turno_do_jogador` e deixe no topo do arquivo, ao lado do `CLASSES`.

Você vai descobrir um problema: as ações precisam do `jogador` e do `inimigo`, que só existem dentro da função. Resolva e explique como.

E4.1 A PERGUNTA REDUNDANTE

Depende de: capítulo 4.

O `agir` do Dragão pergunta `if self.habilidade_disponivel()` antes de chamar o `usar_habilidade`, que já confere a mesma coisa.

Tire a pergunta e jogue. Alguma coisa mudou na tela? Por quê?

Agora decida se ela deve voltar, e justifique. Dica: pense em quem imprime a mensagem de recarga quando é um **inimigo** que tenta usar a habilidade.

E4.2 A HABILIDADE QUE NÃO EXISTE

Depende de: capítulo 4.

O Goblin não tem habilidade especial e herda o `efeito_da_habilidade` padrão, que só avisa isso.

Responda: por que a mãe tem um efeito padrão que não faz nada, em vez de simplesmente não ter o método? O que aconteceria com o menu se ela não tivesse?

E5.1 UM TERCEIRO BÔNUS

Depende de: capítulo 5.

Crie um item que dê bônus temporário de **vida máxima**, usando a classe `BonusTemporario`.

Conte quantas linhas você precisou escrever. Compare com o que teria sido de manhã, quando cada bônus exigia dois atributos e dois blocos.

E5.2 LEITURA DIFÍCIL

Depende de: capítulo 5.

Este extra é de leitura. Procure o termo **code smell** — "cheiro de código".

Dois cheiros clássicos têm nome próprio e apareceram hoje: *duplicated code* e *data clumps* (grupos de dados que andam juntos).

Responda: em qual capítulo cada um deles apareceu, e por que se usa a palavra "cheiro" em vez de "erro"?

Checklist de entrega

CONTINUA FUNCIONANDO

- ☐ O jogo roda do início ao fim, seis fases, sem erro na tela.
 - ☐ O Guerreiro nunca erra o ataque; os outros erram de vez em quando.
 - ☐ As quatro opções do turno funcionam, e opção inválida não trava.
 - ☐ As habilidades entram em recarga e voltam no tempo certo.
 - ☐ Os seis inimigos continuam decidindo como decidiam ontem.
 - ☐ Elixir e tônico duram três turnos e voltam ao normal avisando.
 - ☐ Nenhuma mensagem da tela mudou de texto.
-

CAPÍTULOS 1 E 2

- ☐ Existe `calcular_dano` em `Personagem`, e ele usa `return`.
 - ☐ `Guerreiro.atacar` não tem mais `random.randint` escrito à mão.
 - ☐ Existe a função `recompensar` e o `main` ficou menor.
 - ☐ Não sobrou nenhum `20`, `5` ou `3` solto no `atacar`.
 - ☐ Não sobrou nenhum `0.3`, `0.4` ou `0.5` solto nos `agir`.
 - ☐ As constantes estão em MAIÚSCULAS e no topo do arquivo.
-

CAPÍTULOS 3 E 4

- ☐ `turno_do_jogador` não tem mais cadeia de `elif`.
 - ☐ O menu é impresso a partir do dicionário, não de `print` soltos.
 - ☐ Existe `usar_habilidade` em `Personagem`.
 - ☐ Nenhuma filha confere a recarga por conta própria.
 - ☐ Nenhuma filha chama `iniciar_recarga`.
 - ☐ Cada habilidade avisa a recarga com o mesmo verbo de antes.
 - ☐ A palavra `habilidade_especial` não existe mais em nenhum arquivo.
-

CAPÍTULO 5

- ☐ Existe a classe `BonusTemporario` no `personagem.py`.
- ☐ Ela não sabe nada sobre ataque, defesa ou personagem.
- ☐ Os quatro atributos `_bonus_*` e `_turnos_bonus_*` não existem mais.
- ☐ Buscar por `_bonus_` nos quatro arquivos não encontra nada.
- ☐ Cada bloco do `iniciar_turno` cabe em quatro linhas.
- ☐ Orc e Troll perguntam `bonus_ataque.ativo()` em vez de ler atributo.

ORGANIZAÇÃO

- ☐ Exatamente quatro arquivos na pasta. Nenhum arquivo novo entrou hoje.
- ☐ A pasta `rpg-antes/` foi apagada depois de conferir tudo.
- ☐ Não sobrou código comentado das versões antigas.
- ☐ O código roda com `python main.py`, sem editar nada antes.

O ÚLTIMO TESTE

Abra o `git diff` do dia — ou compare com a pasta `rpg-antes/` — e conte as linhas. Você deve ter **apagado mais linhas do que escreveu**.

Foi o único dia do curso em que isso vai acontecer, e o jogo faz exatamente a mesma coisa que fazia de manhã.